Chapter 2.
Time and Motion



Contents

2.1 Smooth One-Dimensional Trajectories
2.2 Multi-Axis Trajectories
2.3 Multi-Segment Trajectories
2.4 Interpolation of Orientation in 3D
2.5 Cartesian Motion in 3D
2.6 Application: Inertial Navigation

- In robotics, we often need to generate a pose that varies smoothly with time from ξ_0 to ξ_1.
- We require position and orientation to vary smoothly with time.
- Smoothness in this context means that its first few temporal derivatives of position and orientation are continuous.
- Typically, velocity and acceleration are required to be continuous and sometimes also the derivative of acceleration or jerk.

DRAFT

- A path is a spatial construct – a locus in space that leads from an initial pose to a final pose.
- A trajectory is a path with specified timing.
- For example there is a path from A to B, but there is a trajectory from A to B in $10\ s$ or at $2\ m/s$.
- The first problem is to construct a smooth path from ξ_0 to ξ_1 which could be defined by some function $\xi(s), s \in [0, 1]$.
- The second problem is to construct a smooth trajectory, that is smooth motion along the path which requires that $s(t)$ is a smooth function of time.

DRAFT

- We start by discussing how to generate smooth trajectories in one dimension.
- We then extend that to the multi-dimensional case and then to piecewise-linear trajectories that visit a number of intermediate points without stopping.

DRAFT

2.1 Smooth One-Dimensional Trajectories

- We start our discussion with a scalar function of time that has a specified initial and final value.
- An obvious candidate for such a smooth function is a polynomial function of time.
- Polynomials are simple to compute and can easily provide the required smoothness and boundary conditions.

DRAFT

- A quintic (fifth-order) polynomial is commonly used

$$q(t) = At^5 + Bt^4 + Ct^3 + Dt^2 + Et + F$$

where time $t \in [0, T]$. The first- and second-derivatives are also polynomials

$$\dot{q}(t) = 5At^4 + 4Bt^3 + 3Ct^2 + 2Dt + E$$
$$\ddot{q}(t) = 20At^3 + 12Bt^2 + 6Ct + 2D$$

and therefore smooth. The third-derivative, jerk, will be a quadratic.

DRAFT

- The trajectory has defined boundary conditions for position (q_0, q_T), velocity $(\dot{q}_0, \dot{q}_T)$ and acceleration $(\ddot{q}_0, \ddot{q}_T)$.
- For the boundary conditions $t = 0$ and $t = T$, we obtain six equations which can be written in matrix form as

$$\begin{pmatrix} q_0 \\ q_T \\ \dot{q}_0 \\ \dot{q}_T \\ \ddot{q}_0 \\ \ddot{q}_T \end{pmatrix} = \begin{pmatrix} 0 & 0 & 0 & 0 & 0 & 1 \\ T^5 & T^4 & T^3 & T^2 & T & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 5T^4 & 4T^3 & 3T^2 & 2T & 1 & 0 \\ 0 & 0 & 0 & 2 & 0 & 0 \\ 20T^3 & 12T^2 & 6T & 2 & 0 & 0 \end{pmatrix} \begin{pmatrix} A \\ B \\ C \\ D \\ E \\ F \end{pmatrix}$$

- The matrix is square and, if $T \neq 0$, we can invert it to solve for the coefficient vector (A, B, C, D, E, F).

DRAFT

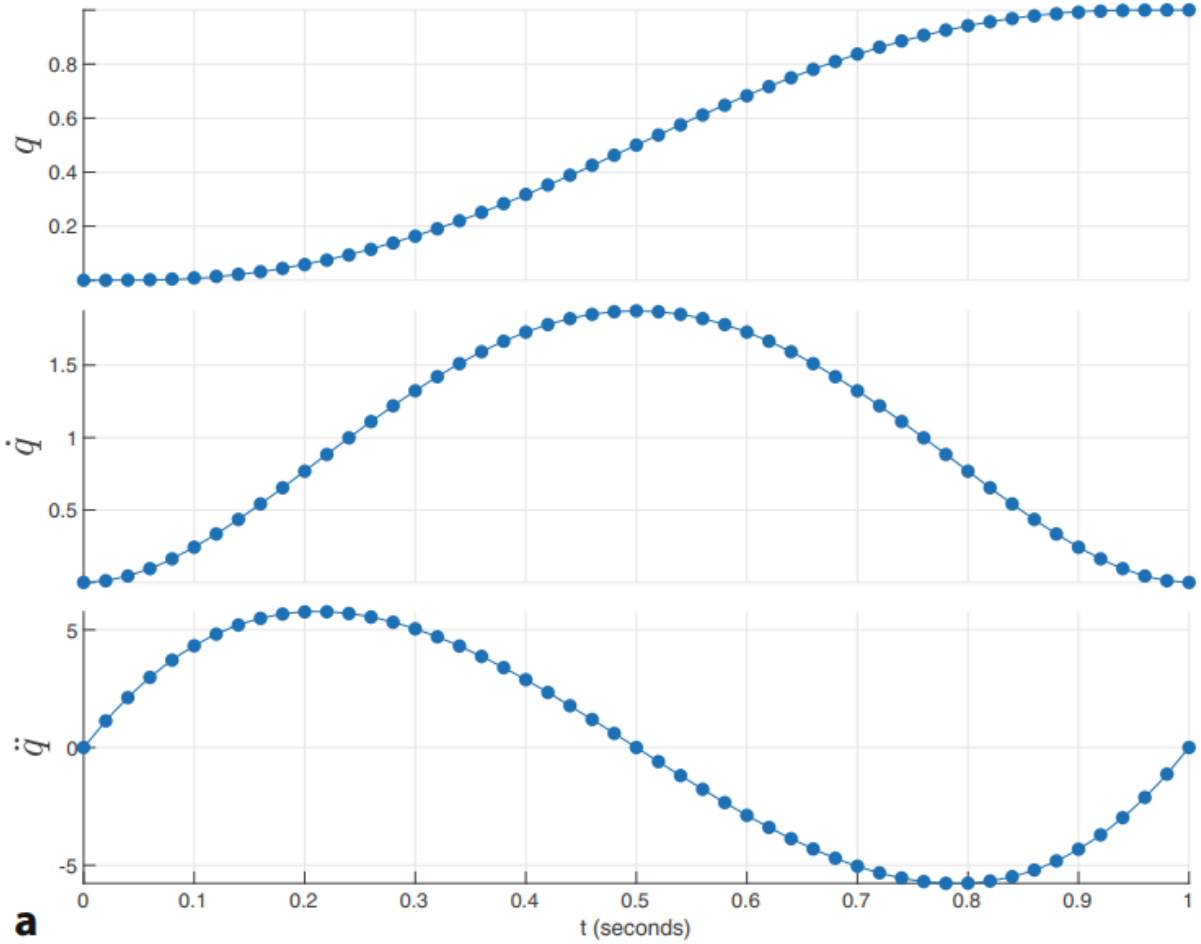
- We can generate the trajectory using a quintic polynomial

```
>> t = linspace(0,1,50); % 0 to 1 in 50 steps
>> [q,qd,qdd] = quinticpolytraj([0 1],[0 1],t);
>> stackedplot(t,[q' qd' qdd'])
```

- For most of the time, the velocity is far less than the maximum.

```
>> mean(qd)/max(qd)
ans =
    0.5231
```

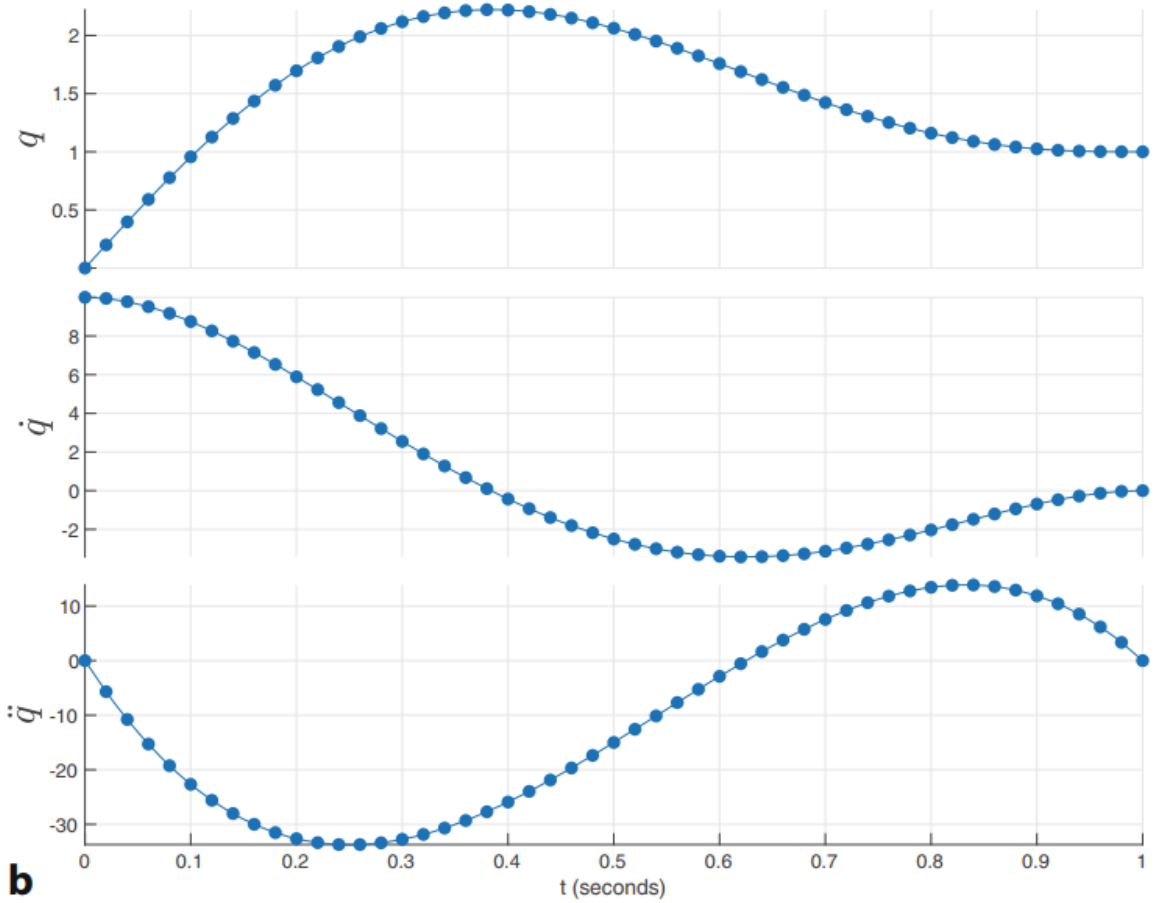
is only 52% of the peak so we are not using the motor as fully as we could.



DRAFT

```
>> [q2,qd2,qdd2] = quinticpolytraj([0 1],[0 1],t, ...
>>   VelocityBoundaryCondition=[10 0]);
```

- This illustrates an important problem with polynomials – nonzero initial velocity causes the polynomial to overshoot the terminal value.

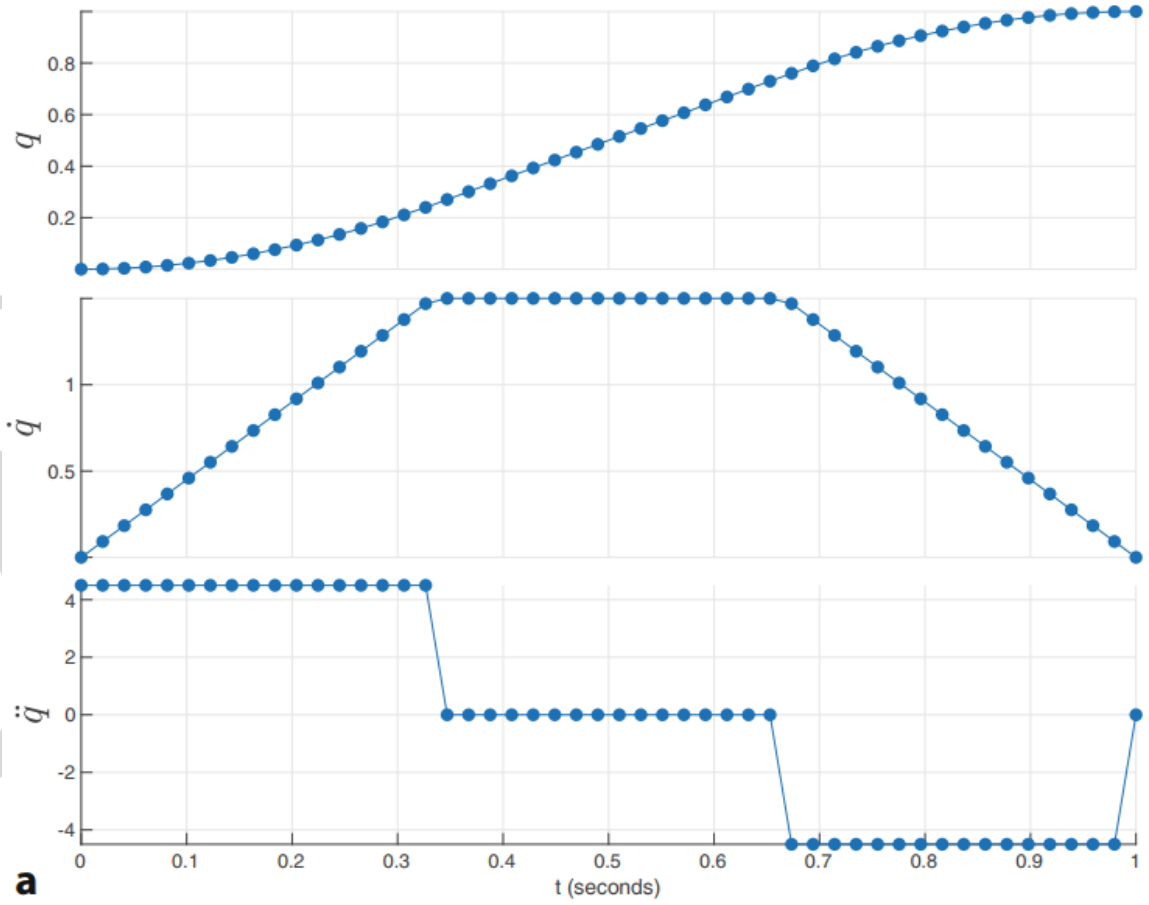


- A real robot joint has a well-defined maximum speed and, for minimum-time motion, we want to be operating at that maximum for as much of the time as possible.
- We would like the velocity curve to be flatter on top.

```
>> [q,qd,qdd] = trapveltraj([0 1],50);
>> stackedplot(t,[q' qd' qdd'])
```

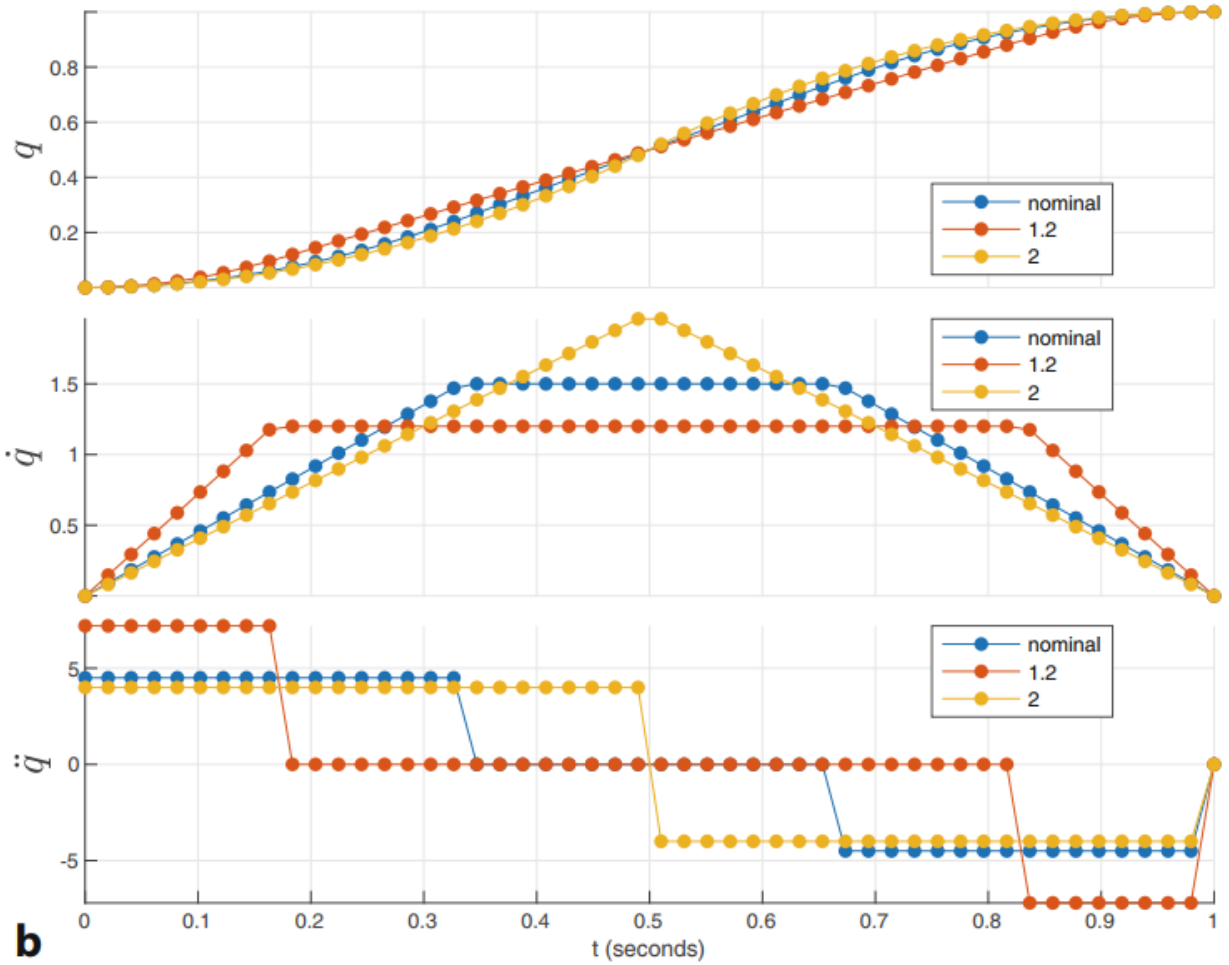
DRAFT

- The velocity trajectory has a trapezoidal shape.
- The corresponding segments of the position trajectory are a straight line with parabolic blends.
- This type of trajectory is commonly used in industrial motor drives.



```
>> [q2,qd2,qdd2] = trapveltraj([0 1],50,EndTime=1,PeakVelocity=1.2);
>> [q3,qd3,qdd3] = trapveltraj([0 1],50,EndTime=1,PeakVelocity=2);
```

- We see that as the velocity of the linear segment increases, its duration decreases and ultimately its duration would be zero.



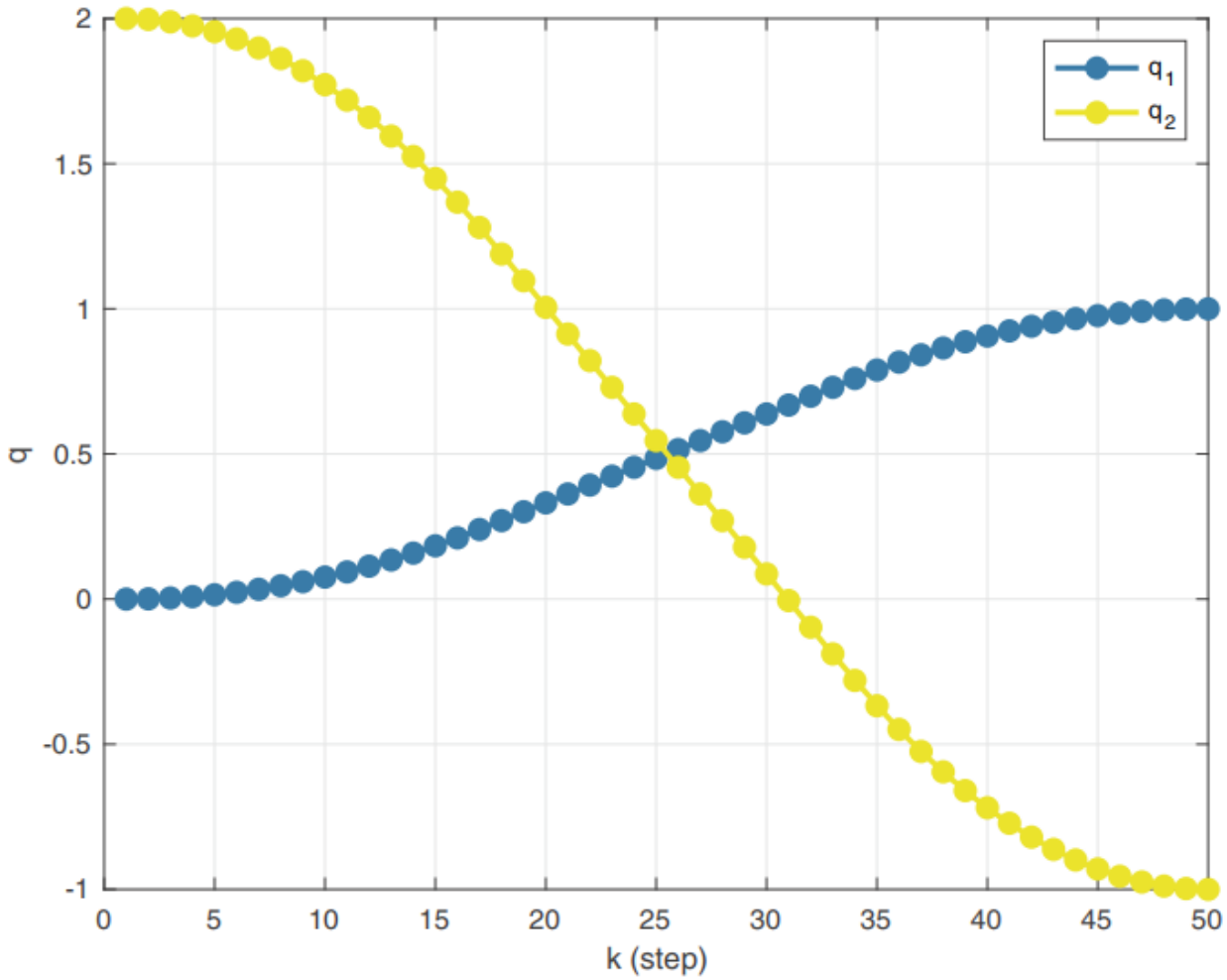
2.2 Multi-Axis Trajectories

- Most useful robots have more than one axis of motion and it is quite straightforward to extend the smooth scalar trajectory to the vector case.
- In terms of configuration space, these axes of motion correspond to the dimensions of the robot's configuration space – to its degrees of freedom.
- For example, the configuration vector of wheeled mobile robot might be its position $q = (x, y)$ or its position and heading angle $q = (x, y, \theta)$.
- We would require smooth multi-dimensional motion from an initial configuration vector to a final configuration vector

DRAFT

- Consider a 2-axis xy-type robot moving from configuration (0, 2) to (1,-1) in 50 steps with a trapezoidal profile.

```
>> q0 = [0 2]; qf = [1 -1];
>> q = trapveltraj([q0' qf'],50,EndTime=1);
```



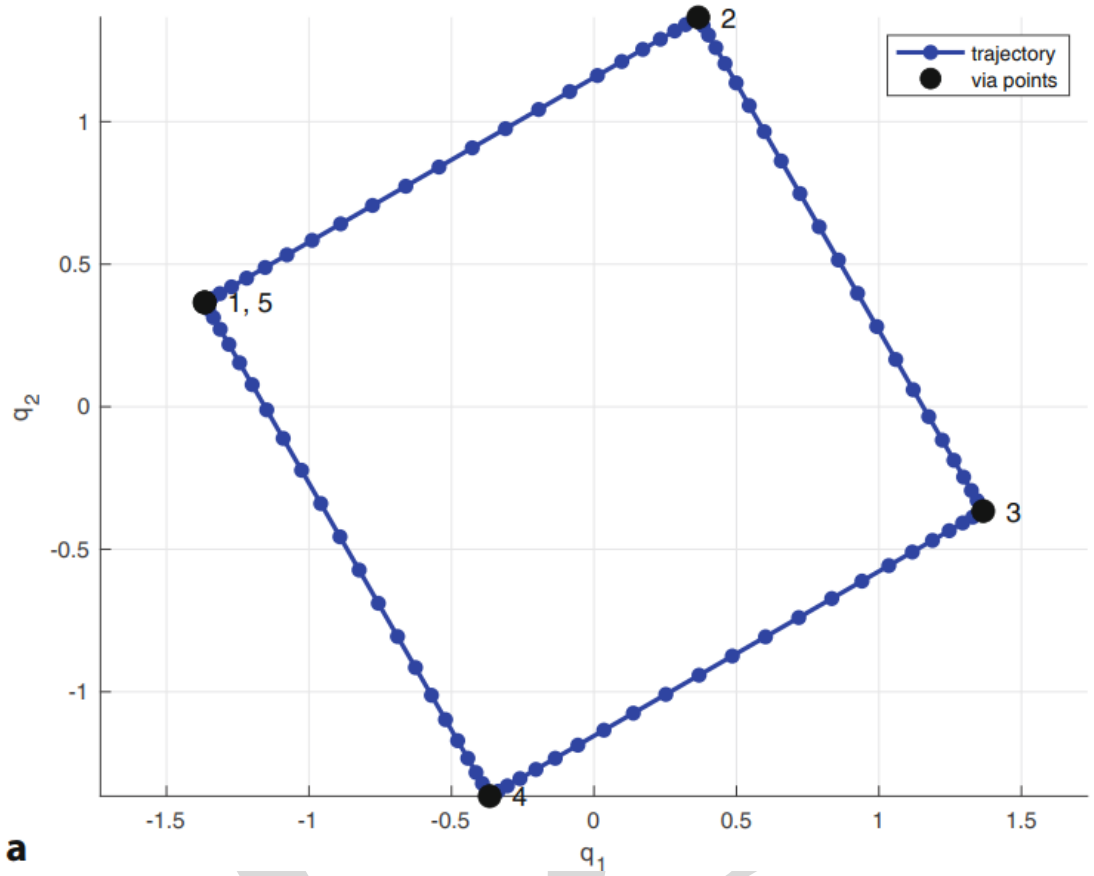
2.3 Multi-Segment Trajectories

- In robotics applications, there is often a need to move smoothly along a path through one or more intermediate or via points or waypoints without stopping.
- Consider that the trajectory is defined by M configurations q_k, so there will be M-1 motion segments.
- Consider again a 2-axis xy-type robot following a path defined by the corners of a rotated square.

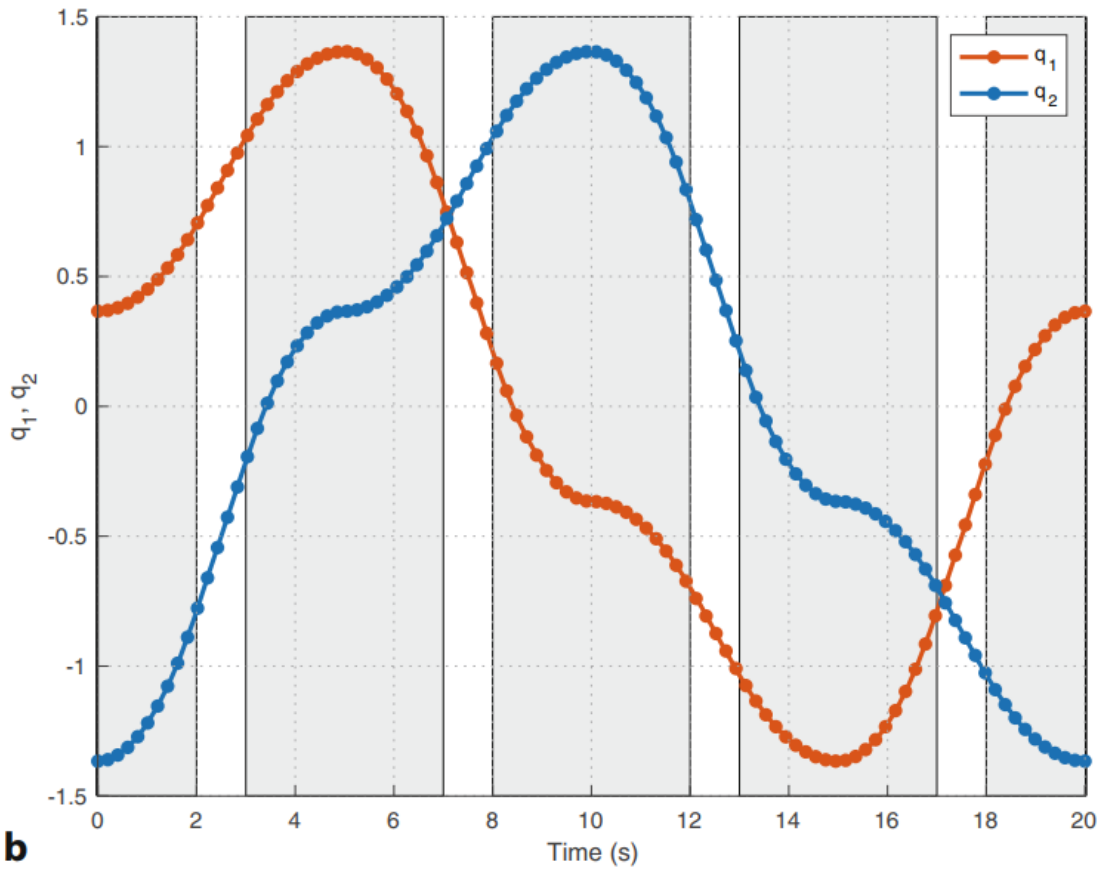
```
>> cornerPoints = [-1 1; 1 1; 1 -1; -1 -1; -1 1];
>> R = so2(deg2rad(30),"theta");
>> via = R.transform(cornerPoints)';
>> [q,qd,qdd,t] = trapveltraj(via,100,EndTime=5);

>> q2 = trapveltraj(via,100,EndTime=5,AccelTime=2);
```

DRAFT



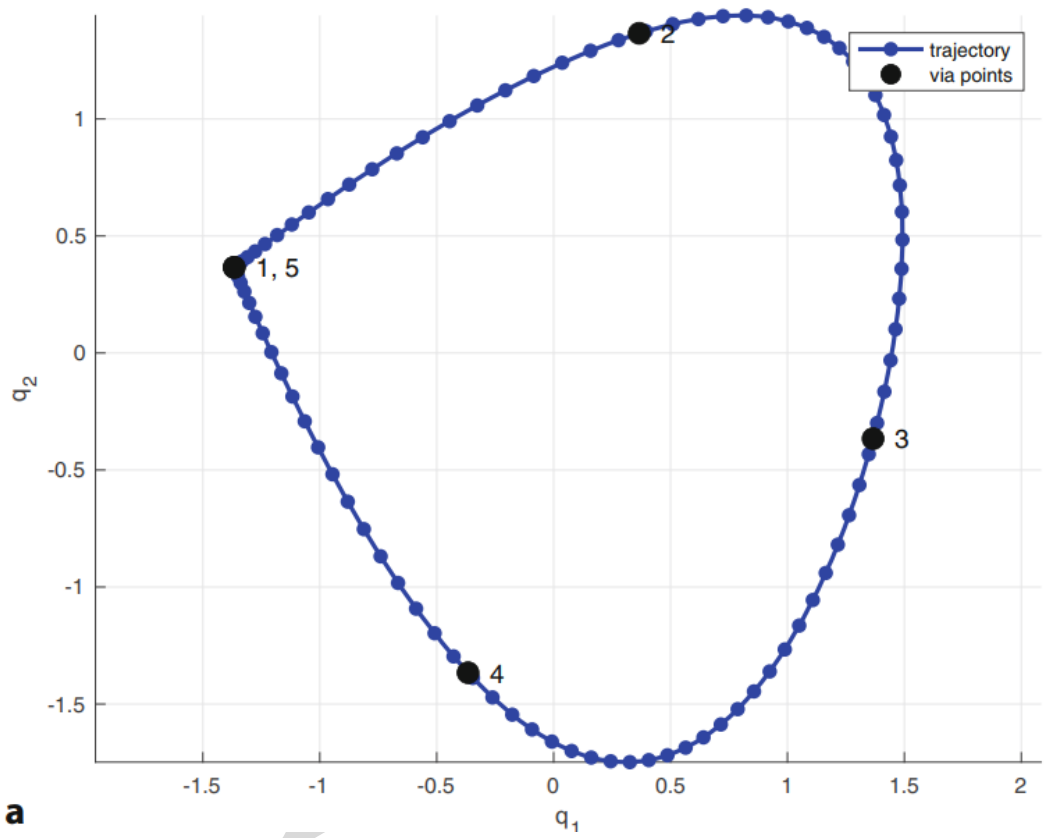
a

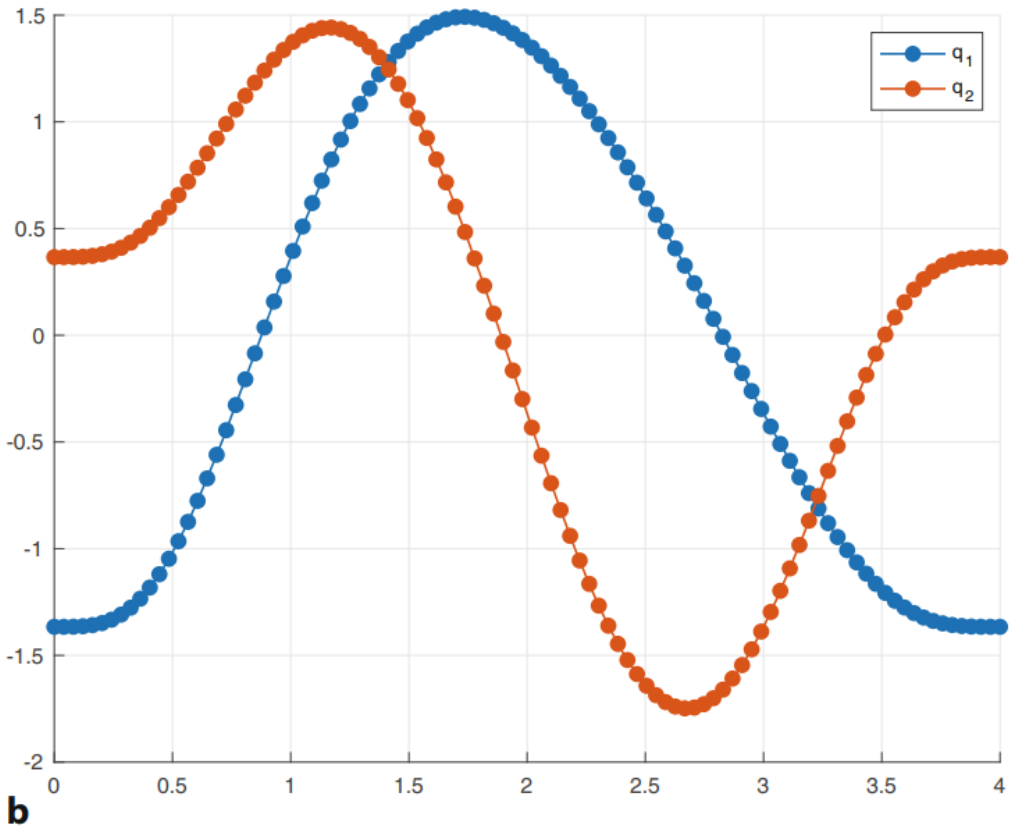


b

- For a continuous trajectory that visits all the via points without stopping we surrender the ability to move along straight line segments.
- Such a trajectory, that minimizes jerk, is given by

```
>> [q,qd,qdd] = minjerkpolytraj(via,[1 2 3 4 5],500);
>> plot(q(1,:),q(2,:),".-")
```





2.4 Interpolation of Orientation in 3D

- In robotics, we often need to interpolate orientation, for example, we require the end effector of a robot to smoothly change from orientation ξ_0 to ξ_1 in $\mathbf{S}^3$.
- We require some function

$$\xi(s) = \sigma(\xi_0, \xi_1, s),$$

where $s \in [0, 1]$, with boundary conditions

$$\sigma(\xi_0, \xi_1, 0) = \xi_0 \text{ and } \sigma(\xi_0, \xi_1, 1) = \xi_1,$$

and where $\sigma(\xi_0, \xi_1, s)$ varies smoothly for intermediate values of s.

DRAFT

- A workable and commonly used approach is to consider a 3-parameter representation such as Euler or roll-pitch-yaw angles or exponential coordinates and use linear interpolation

$$\sigma(\boldsymbol{\Gamma}_0,\ \boldsymbol{\Gamma}_1,\ s) = (1-s)\boldsymbol{\Gamma}_0 + s\boldsymbol{\Gamma}_1, s \in [0,1]$$

and converting the interpolated vector back to a rotation matrix.

- For example, we define two orientations and create a trajectory between them with 50 uniform steps

```
>> rpy0 = [-1 -1 0]; rpy1 = [1 1 0];

>> rpy = quinticpolytraj([rpy0' rpy1'],[0 1],linspace(0,1,50));
```

- For large orientation change we see that the axis, around which the coordinate frame rotates, changes along the trajectory. The motion, while smooth, can look a bit uncoordinated.

DRAFT

- Interpolation of unit quaternions is only a little more complex than for 3-angle vectors and results in a rotation about a fixed axis in space.
- We first find the two equivalent unit quaternions

```
>> q0 = quaternion(eul2rotm(rpy0),"rotmat","point");
>> q1 = quaternion(eul2rotm(rpy1),"rotmat","point");
```

and then interpolate them in 50 uniform steps

```
>> q = q0.slerp(q1,linspace(0,1,50));
```

- Quaternion interpolation is achieved using spherical linear interpolation.
- This is constant velocity motion with a discontinuity at the start and end points. For continuous motion we can interpolate using trapezoidal or cubic time scaling.

DRAFT

- Interpolation of unit quaternions is only a little more complex than for 3-angle vectors and results in a rotation about a fixed axis in space.
- We first find the two equivalent unit quaternions

```
>> q0 = quaternion(eul2rotm(rpy0),"rotmat","point");
>> q1 = quaternion(eul2rotm(rpy1),"rotmat","point");
```

and then interpolate them in 50 uniform steps

```
>> q = q0.slerp(q1,linspace(0,1,50));
```

- Quaternion interpolation is achieved using spherical linear interpolation.
- This is constant velocity motion with a discontinuity at the start and end points. For continuous motion we can interpolate using trapezoidal or cubic time scaling.

DRAFT

2.5 Cartesian Motion in 3D

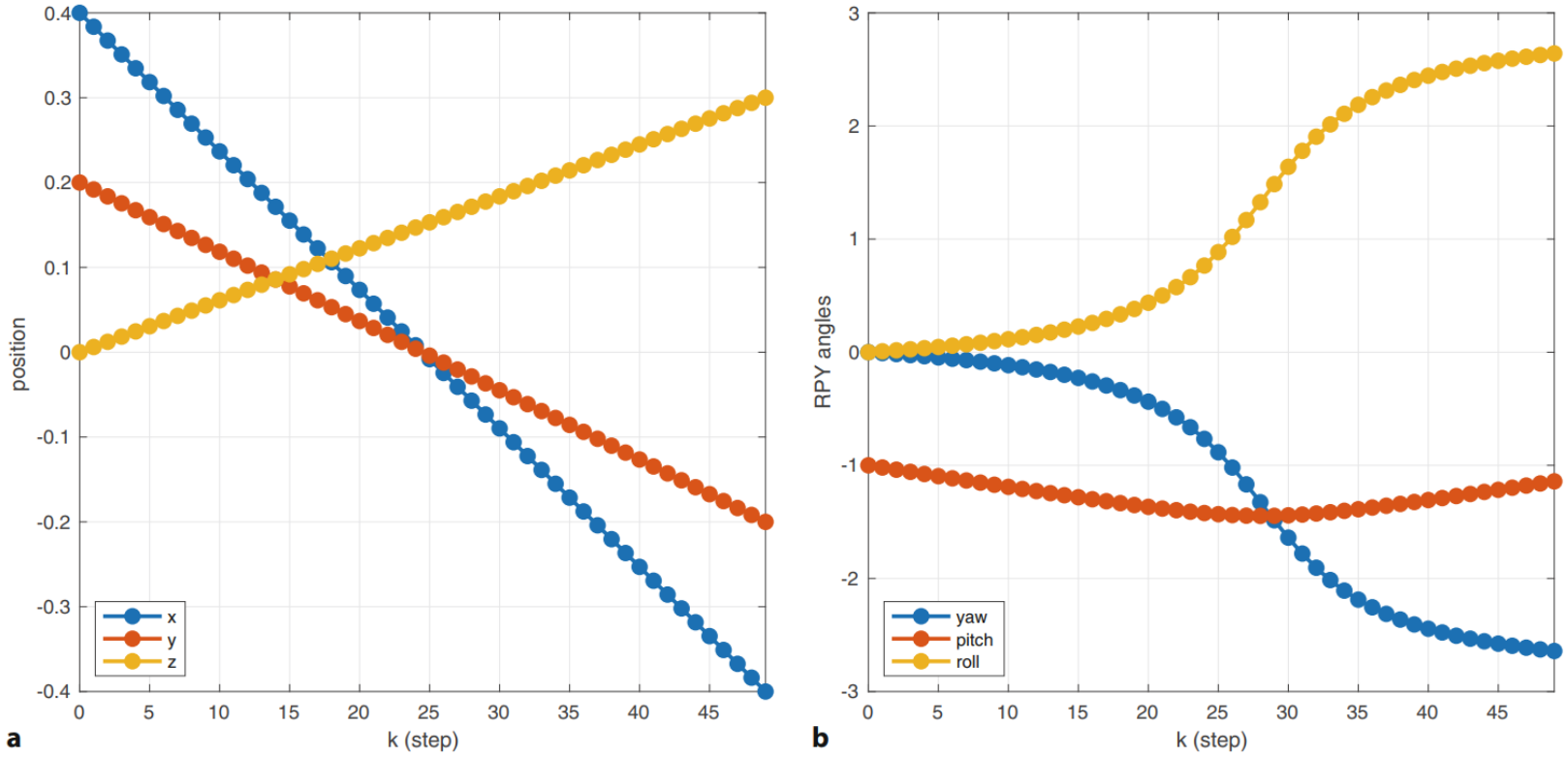
- Another common requirement in robotics is to find a smooth path between two 3D poses in $\mathbb{R}^3 \times \mathbf{S}^3$ which involves change in position as well as in orientation.
- In robotics, this is often referred to as Cartesian motion.
- We represent the initial and final poses as $\mathbf{SE}(3)$ matrices

```
>> T0 = se3(eul2rotm([1.5 0 0]),[0.4 0.2 0]);
>> T1 = se3(eul2rotm([-pi/2 0 -pi/2]),[-0.4 -0.2 0.3]);
```

- The translational component is linearly interpolated, while rotation is interpolated using unit quaternion spherical linear interpolation as introduced above.

DRAFT

- We see that the position coordinates vary linearly with time, but that the roll-pitch-yaw angles are nonlinear with time and that has two causes.
- Firstly, roll-pitch-yaw angles are a nonlinear transformation of the linearly-varying quaternion orientation.
- Secondly, this particular trajectory passes very close to the roll-pitchyaw singularity.



2.6 Application: Inertial Navigation

- An inertial navigation system or INS is a self contained unit that estimates its velocity, orientation and position by measuring accelerations and angular velocities and integrating them over time.
- Importantly, it has no external inputs such as radio signals from satellites.



- An INS estimates its pose with respect to an inertial reference frame which is typically denoted as {0}.
- The frame typically has its z-axis upward or downward and the x- and y-axes establish a locally horizontal tangent plane.
- The coordinate frame {B} is attached to the moving vehicle or robot and is known as the body- or body-fixed frame.

DRAFT

2.6.1 Gyroscopes

- Any sensor that measures the rate of change of orientation is known, for historical reasons, as a gyroscope.
- It typically consists of a spinning disk mounted in a gimbal frame.
- A spinning rotor resists changes in its orientation due to angular momentum.

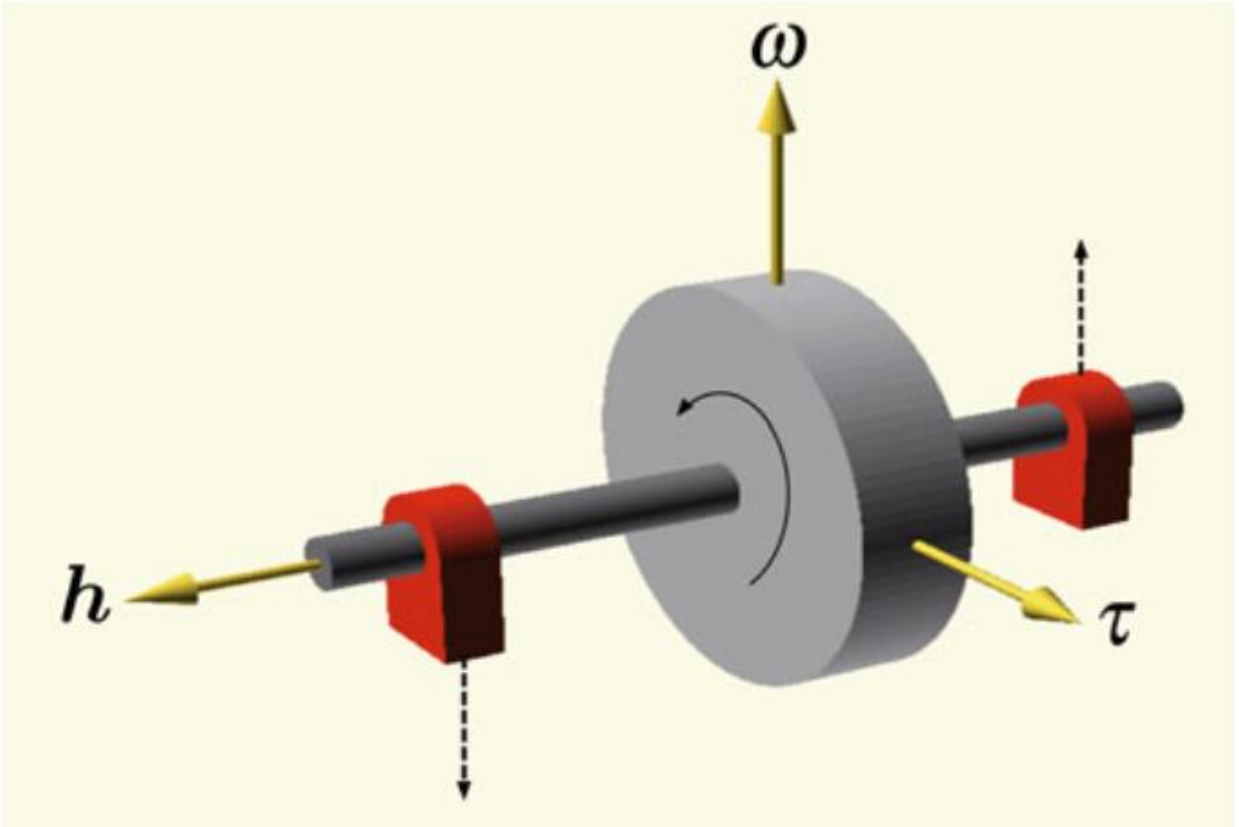
DRAFT

- Described mathematically by the equation:

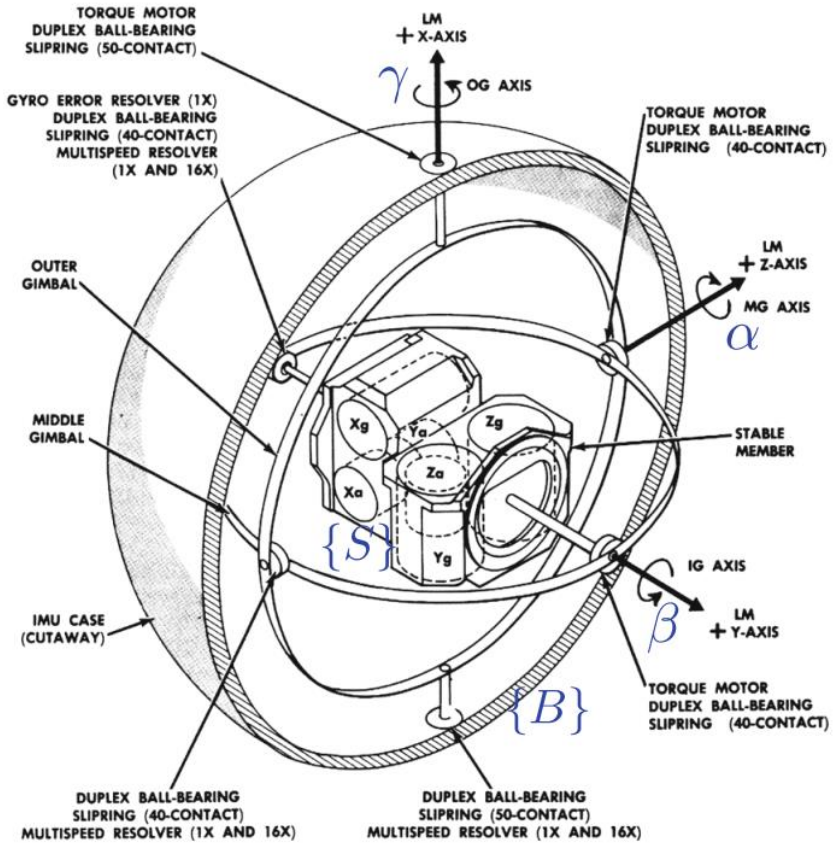
$$\boldsymbol{\tau} = \boldsymbol{\omega} \times \boldsymbol{h},$$

where $\boldsymbol{\tau}$ – torque; $\boldsymbol{\omega}$ – angular velocity; $\boldsymbol{h}$ – angular moment.

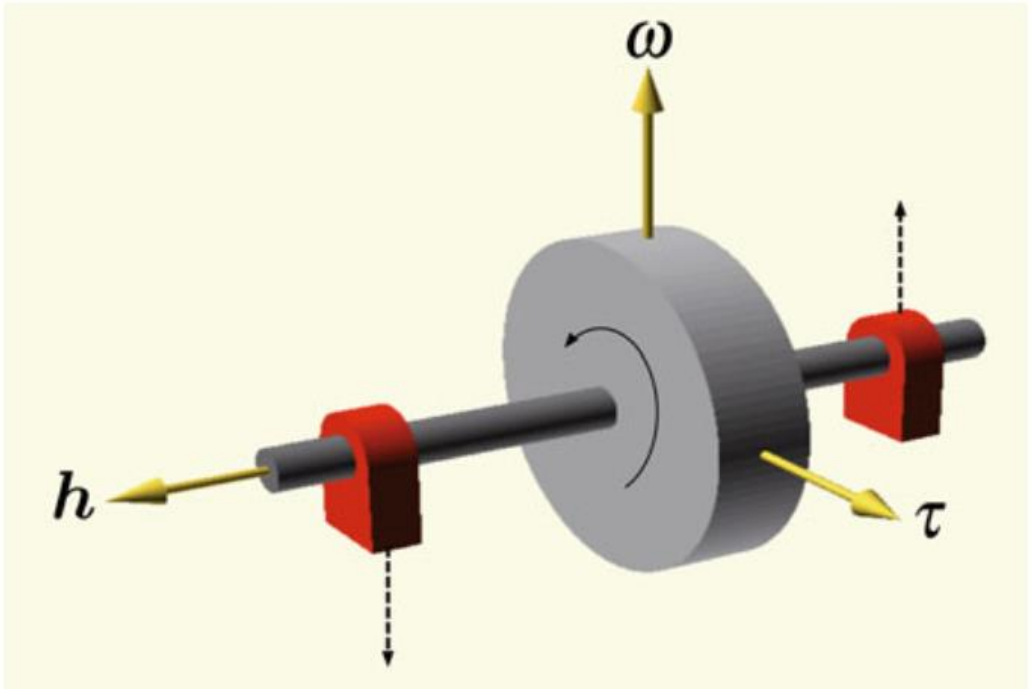
- It is the cross product in this equation that makes the gyroscope move in a contrary way.



- Early spacecraft used gimballed gyroscopes to maintain a stable orientation in inertial space.
- The gimbals allowed the spacecraft to rotate about the stable platform, and the relative orientation was determined by measuring the gimbal angles.



- Alternatively, we can fix the gyroscope to the vehicle in the strapdown configuration.
- If the vehicle rotates with an angular velocity $\boldsymbol{\omega}$, the attached gyroscope will precess and exert an orthogonal torque $\boldsymbol{\tau}$ which can be measured.
- If the magnitude of $\boldsymbol{h}$ is high, then this kind of sensor is very sensitive – a very small angular velocity leads to an easily measurable torque.



DRAFT

- Optical gyroscopes (e.g., RLG, FOG) are very accurate but expensive and bulky.
- MEMS gyroscopes in smartphones and drones are compact and based on vibrating elements. Rotation about a specific axis causes orthogonal displacements that are sensed electronically.
- Combine three orthogonal sensors to measure full 3D angular velocity.

DRAFT

- If the orientation of the sensor frame is initially ξ_B, then the evolution of estimated orientation can be written in discrete-time form as

$$\hat{\xi}_{B\langle k+1\rangle} \leftarrow \hat{\xi}_{B\langle k\rangle} \oplus {}^{B\langle k\rangle}\xi_{B\langle k+1\rangle}$$

where ${}^{B\langle k\rangle}\xi_{B\langle k+1\rangle}$ is the incremental rotation over the time step which can be computed from measured angular velocity.

- It is common to assume that ${}^{B}\boldsymbol{\omega}_B$ is constant over the time interval δ_t. In terms of $\mathbf{SO}(3)$ rotation matrices, the orientation update is

$$\hat{\mathbf{R}}_{B\langle k+1\rangle} \leftarrow \hat{\mathbf{R}}_{B\langle k\rangle}\, \mathrm{e}^{\delta_t [{}^{B}\boldsymbol{\omega}_B^{\#}]_\times}$$

- The result should be periodically normalized to eliminate accumulated numerical error.

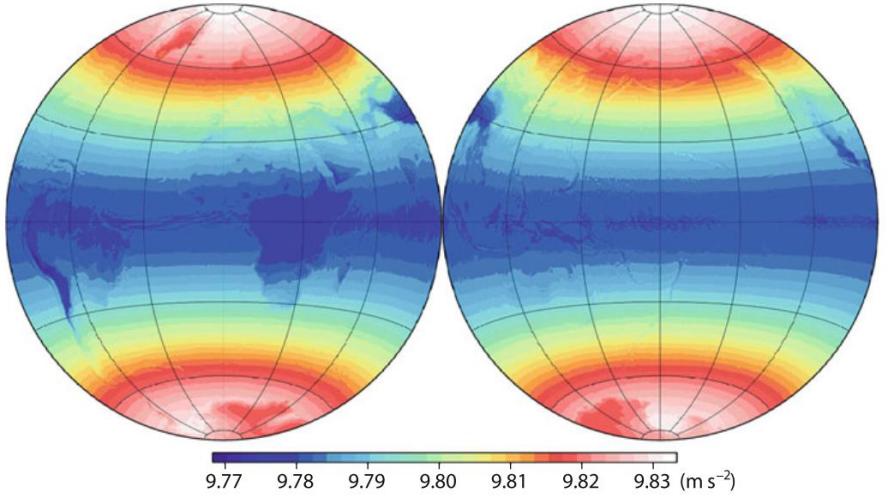
DRAFT

2.6.2 Accelerometers

- Accelerometers are sensors that measure acceleration.
- Even when not moving, they sense the acceleration due to gravity.
- Gravitational acceleration is a function of our distance from the Earth's center, and the material in the Earth beneath us.

$$g \approx 9.780327\left(1 + 0.0053024 \sin^2 \theta - 0.0000058 \sin^2 2\theta\right) - 0.000003086h$$

where θ is the angle of latitude and h is height above sea level.



DRAFT

- A basic accelerometer: a mass-spring system.
- Newton's law:

$$m\ddot{x}_m = F_s - mg \qquad F_s = kd$$

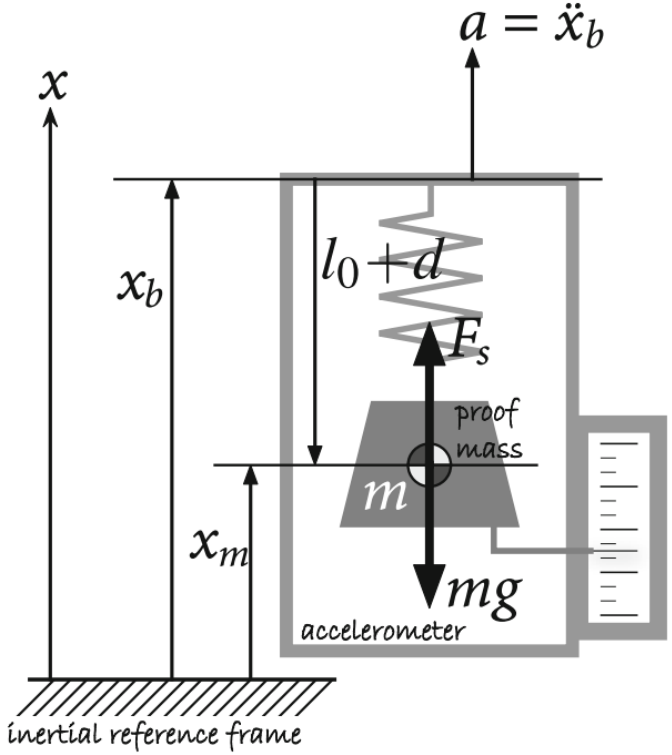
- Displacement relation:

$$x_b - (l_0 + d) = x_m \qquad \ddot{x}_b - \ddot{d} = \frac{1}{m}kd - g$$

- The quantity we wish to measure is the acceleration of the accelerometer: $a = \ddot{x}_b$.
- The output of the sensor is:

$$a^{\#} = a + g$$

- If this accelerometer is stationary, then $a = 0$ yet the measured acceleration would be $a^{\#} = 0 + g = g$ in the upward direction.



DRAFT

- Frame $\{0\}$ with its z-axis vertically upward, and the acceleration is

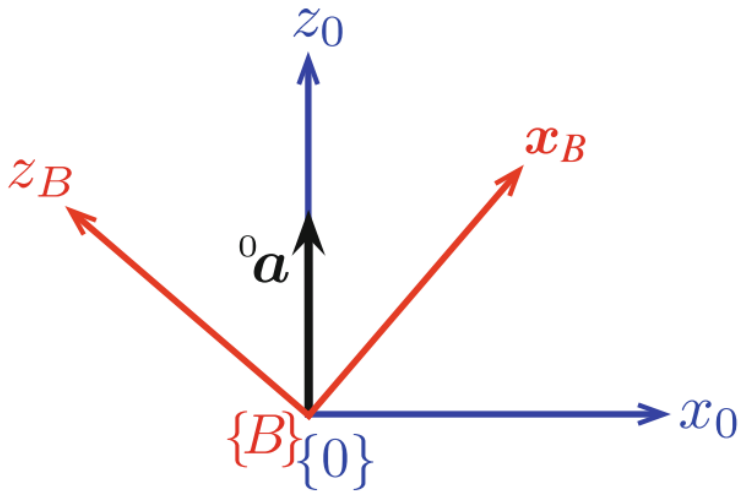
$$ {}^0\boldsymbol{a}_B = \begin{pmatrix} 0 \\ 0 \\ g \end{pmatrix} $$

- In a body-fixed frame $\{B\}$ at an arbitrary orientation expressed in terms of ZYX roll-pitch-yaw angles

$$ {}^0\boldsymbol{\xi}_B = \boldsymbol{\xi}^{r_z}(\gamma) \oplus \boldsymbol{\xi}^{r_y}(\beta) \oplus \boldsymbol{\xi}^{r_x}(\alpha) $$

the gravitational acceleration will be

$$ {}^B\boldsymbol{a}_B = (\ominus\, {}^0\boldsymbol{\xi}_B) \cdot {}^0\boldsymbol{a}_B = \begin{pmatrix} -g \sin\beta \\ g \cos\beta \sin\alpha \\ g \cos\beta \cos\alpha \end{pmatrix} $$



- We can solve for the roll and pitch angles.
- Notice that there is no solution for the yaw angle.

DRAFT

- Frequently, we want to estimate the motion of the vehicle in the inertial frame, and the total measured acceleration in $\{0\}$ is due to gravity and motion

$$ {}^{0}\boldsymbol{a}^{\#} = {}^{0}\boldsymbol{g} + {}^{0}\boldsymbol{a}_{B} $$

- We observe acceleration in the body frame so the acceleration in the world frame is

$$ {}^{0}\hat{\boldsymbol{a}}_{B} = {}^{0}\hat{\mathbf{R}}_{B}\, {}^{B}\boldsymbol{a}_{B}^{\#} - {}^{0}\boldsymbol{g} $$

- Integrating acceleration with respect to time gives the velocity

$$ {}^{0}\hat{\boldsymbol{v}}_{B}(t) = \int_{0}^{t} {}^{0}\hat{\boldsymbol{a}}_{B}(t)\, \mathrm{d}t $$

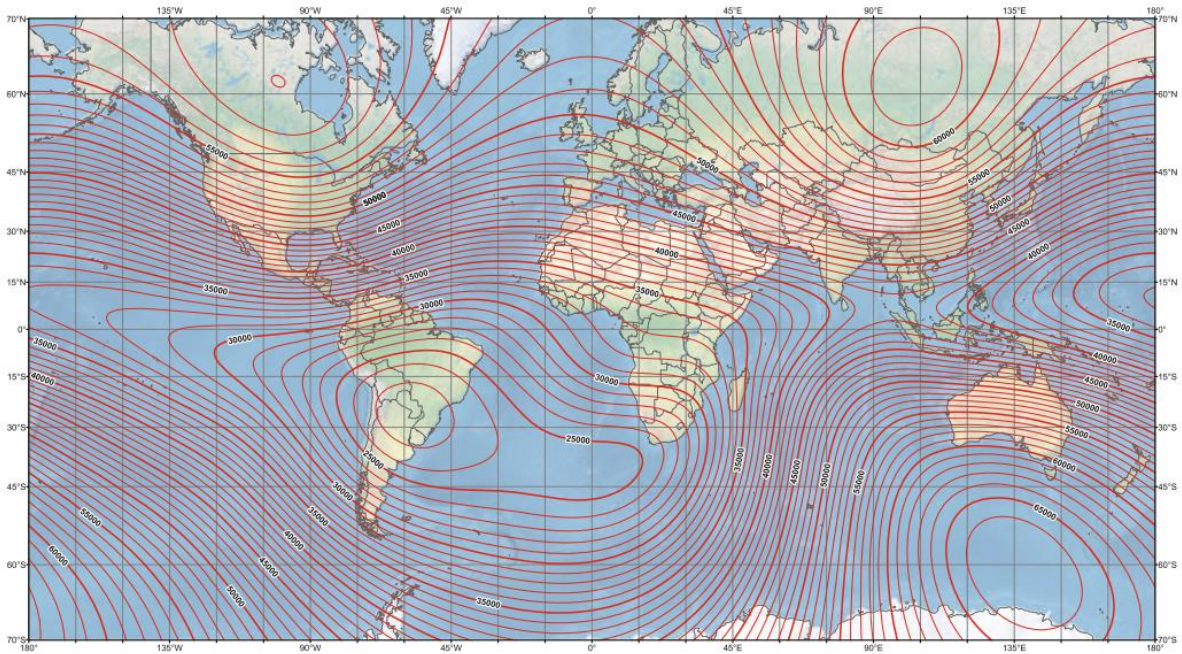
- Integrating again gives its position

$$ {}^{0}\hat{\boldsymbol{p}}_{B}(t) = \int_{0}^{t} {}^{0}\hat{\boldsymbol{v}}_{B}(t)\, \mathrm{d}t $$

DRAFT

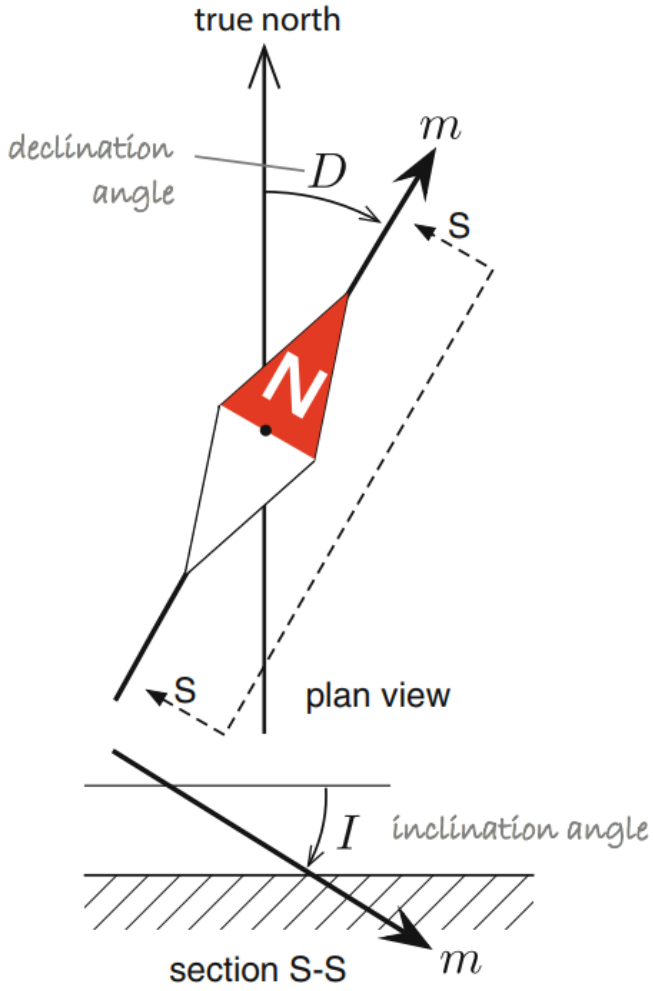
2.6.3 Magnetometers

- The Earth is a massive but weak magnet.
- The poles of this geomagnet are the Earth's north and south magnetic poles.
- At any point on the planet, the magnetic flux lines can be considered a vector $\boldsymbol{m}$.



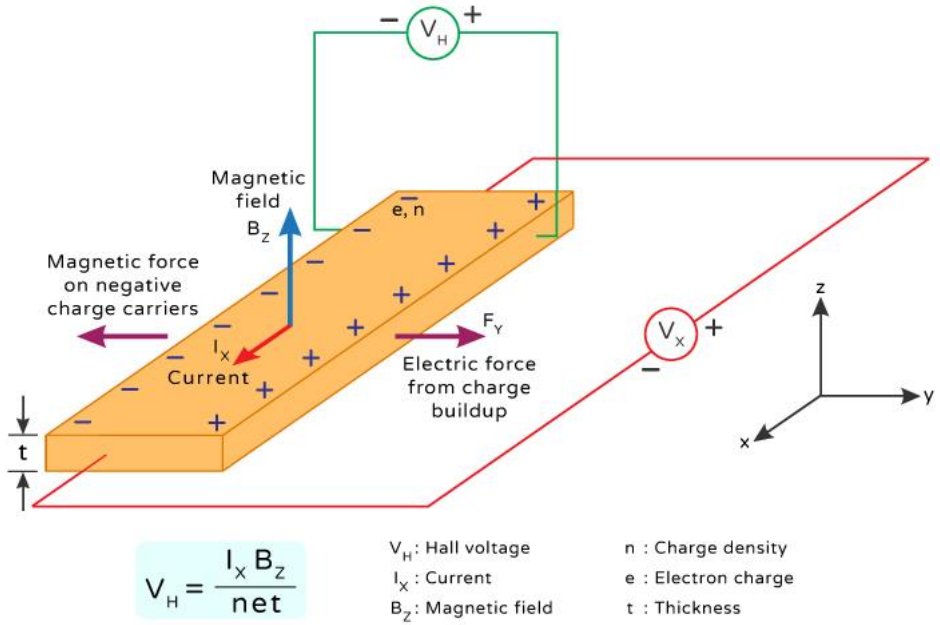
DRAFT

- The vector's direction in terms of two angles: declination and inclination.
- A horizontal projection of the vector $\boldsymbol{m}$ points in the direction of magnetic north and the declination angle D is measured from true north.
- The inclination or dip angle I of the vector is measured in a vertical plane downward from horizontal to $\boldsymbol{m}$.
- The length of the vector, the magnetic flux density, is measured by a magnetometer in units of Tesla (T) and for the Earth this varies from $25-65\ \mu T$.



DRAFT

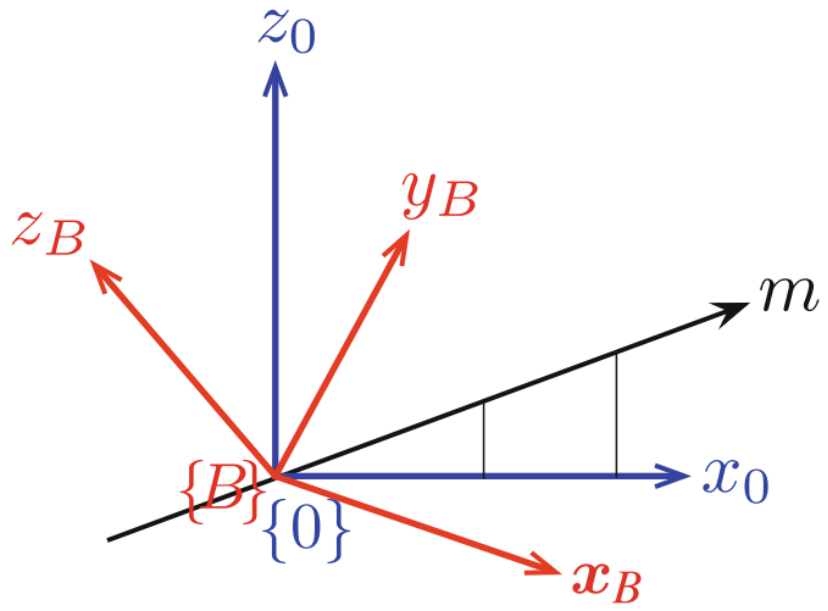
- The key element of most modern magnetometers is a Hall-effect sensor, a semiconductor device which produces a voltage proportional to the magnetic flux density in a direction normal to the current flow.
- Typically, three Hall-effect sensors are packaged together, and arranged so that their sensitive axes are orthogonal.
- The three outputs of such a triaxial magnetometer are the components of the Earth's magnetic flux density vector ${}^{B}\boldsymbol{m}^{\#}$ measured in the body frame $\{B\}$.



DRAFT

- Fig. shows an inertial coordinate frame {0} with its z-axis vertically upward and its x-axis in the horizontal plane and pointing toward magnetic north.
- The magnetic field vector therefore lies in the xz-plane

$$ {}^0\boldsymbol{m} = B \begin{pmatrix} \cos I \\ 0 \\ \sin I \end{pmatrix} $$



DRAFT

- In a body-fixed frame $\{B\}$ at an arbitrary orientation expressed in terms of ZYX roll-pitch-yaw angles

$$ {}^{0}\boldsymbol{\xi}_{B} = \boldsymbol{\xi}^{r_z}(\gamma) \oplus \boldsymbol{\xi}^{r_y}(\beta) \oplus \boldsymbol{\xi}^{r_x}(\alpha) $$

$$ {}^{B}\boldsymbol{m} = (\ominus\, {}^{0}\boldsymbol{\xi}_{B}) \cdot {}^{0}\boldsymbol{m} $$

$$ = \begin{pmatrix} B \sin I \sin\alpha \sin\gamma + \cos\alpha \cos\gamma \sin\beta + B \cos I \cos\beta \cos\gamma \\ B \cos I \cos\beta \sin\gamma - B \sin I \cos\gamma \sin\alpha - \cos\alpha \sin\beta \sin\gamma \\ B \sin I \cos\alpha \cos\beta - B \cos I \sin\beta \end{pmatrix} $$

- We can solve for the yaw angle

$$ \tan\hat{\gamma} = \frac{\cos\beta \left(m_z^{\#} \sin\alpha - m_y^{\#} \cos\alpha \right)}{m_x^{\#} + B \sin I \sin\beta} $$

which is the magnetic heading and related to the true heading by

$$ {}^{\text{tn}}\hat{\gamma} = \hat{\gamma} - D $$

DRAFT

- This assumes that we know the local declination angle as well as the roll and pitch angles α and β.
- The latter could be estimated from acceleration measurements, and many triaxial Hall-effect sensor chips also include a triaxial accelerometer for just this purpose.
- Magnetometers are great in theory but problematic in practice.
- Firstly, our modern world is full of magnets and electromagnets.
- Secondly, many objects in our world contain ferromagnetic materials.
- These distort the geomagnetic field leading to local changes in its direction.

DRAFT

2.6.4 Inertial Sensor Fusion

- An inertial navigation system uses the sensors we have just discussed to determine the pose of a vehicle – its position and its orientation.
- The angular velocity, acceleration and magnetic field sensors are rigidly attached to the vehicle.
- The collection of inertial sensors is referred to as an inertial measurement unit or IMU.
- A 6-axis IMU comprises triaxial gyroscopes and accelerometers while a 9-axis IMU comprises triaxial gyroscopes, accelerometers and magnetometers.

DRAFT

- The output of any sensor $x^{\#}$ – gyroscope, accelerometer or magnetometer – is a corrupted version of the true value x and a common model is

$$x^{\#} = sx + b + \varepsilon$$

where s is the scale factor, b is the offset or bias, and ε is "noise".

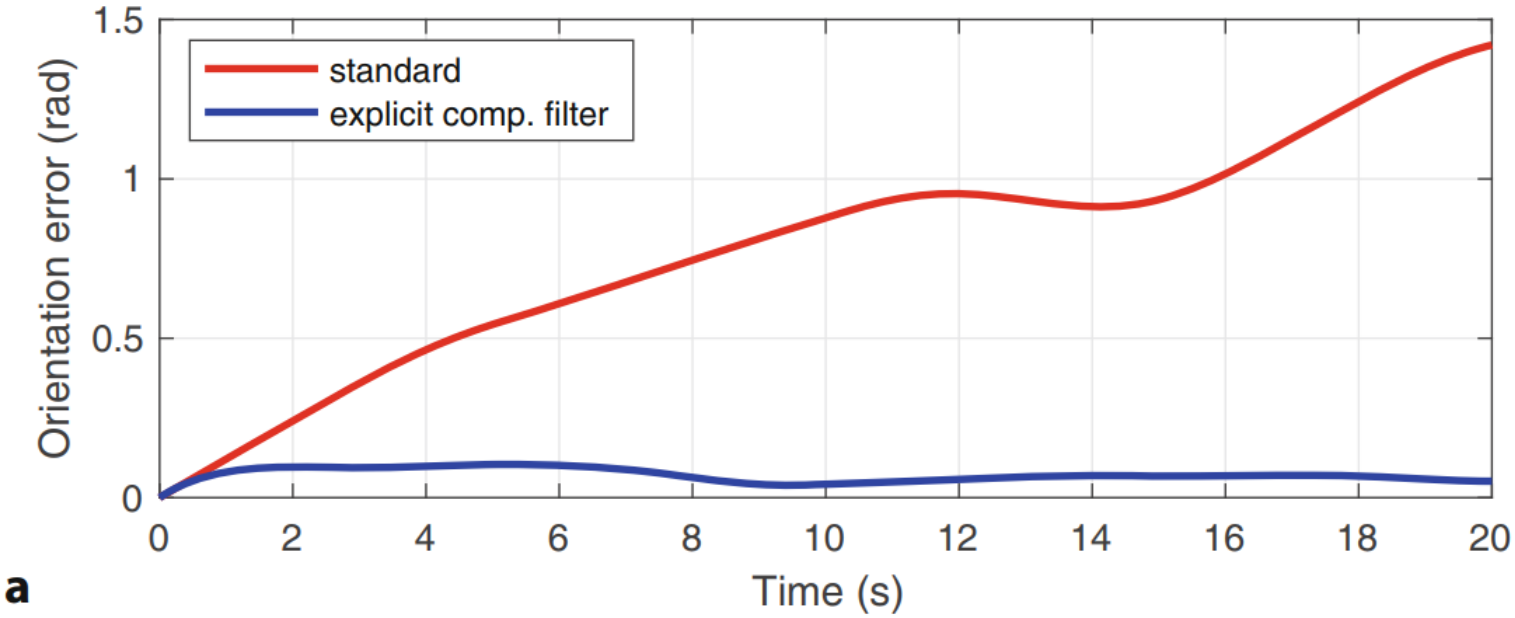
- In practice, bias is the biggest problem because it varies with time and temperature.
- Consider a positive bias on the output of a gyroscopic sensor, the orientation error will grow linearly with time.
- A rule of thumb is that gyroscopes with bias stability of $0.01\,\mathrm{deg}\,h^{-1}$ will lead to position error growing at a rate of $1.85\ km\ h^{-1}$.

DRAFT

- Demonstrate the effect of bias on the estimated orientation:

```
>> [TrueMotion,IMU] = imudata();
>> orientation(1) = quaternion([1 0 0 0]);  % identity quaternion
>> for k = 1:size(IMU.gyro,1)-1
>>   orientation(k+1) = orientation(k) ...
>>                         *quaternion(IMU.gyro(k,:)*IMU.dt,"rotvec");
>> end
```

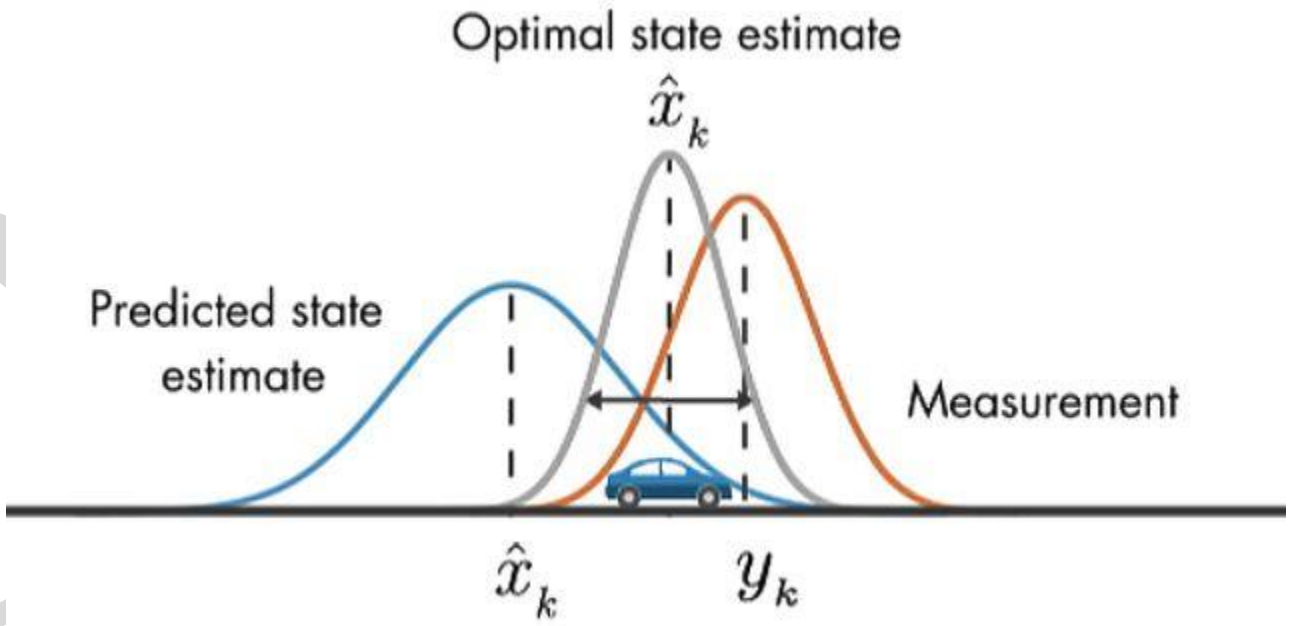
- We can clearly see growth in angular error over time due to bias on the sensor signals.



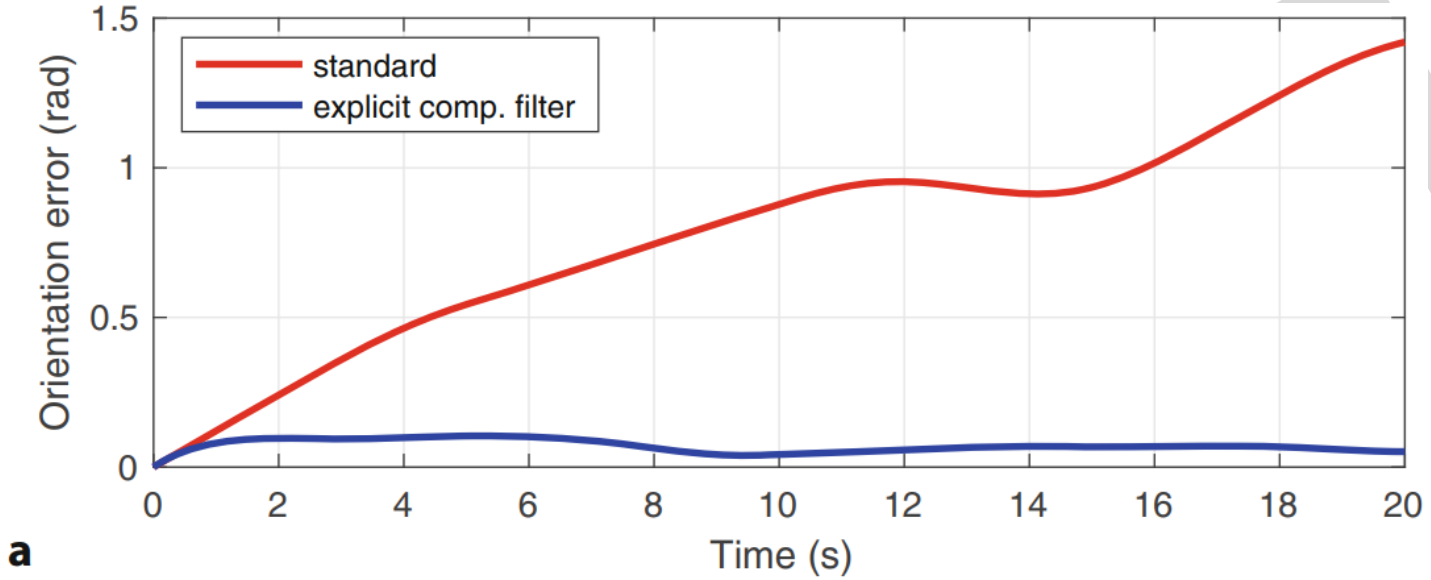
- If we knew the bias, we could subtract it from the sensor measurement before integration.
- This is really only valid over a short time period because the bias is not constant.
- A more sophisticated approach is to estimate the bias online but, to do this, we need to combine information from different sensors – an approach known as sensor fusion.
- We rely on the fact that different sensors have complementary characteristics.

DRAFT

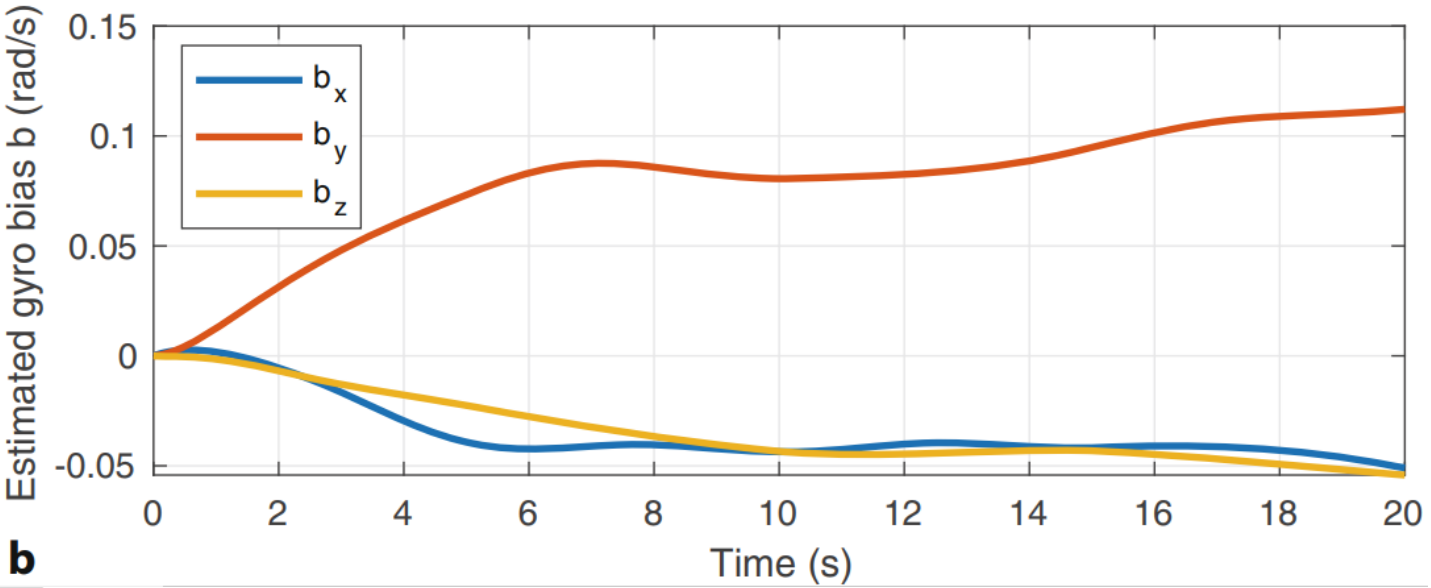
- A common approach is to use an estimation tool called an extended Kalman filter.
- Given a full nonlinear mathematical model that relates the sensor signals and their biases to the vehicle pose and knowledge about the noise (uncertainty) on the sensor signals, the filter gives an optimal estimate of the pose and bias that best explain the sensor signals.



DRAFT


standard
explicit comp. filter
Orientation error (rad)
1.5
1
0.5
0
0
2
4
6
8
10
12
14
16
18
20
Time (s)

a


b x
b y
b z
Estimated gyro bias b (rad/s)
0.15
0.1
0.05
0
-0.05
0
2
4
6
8
10
12
14
16
18
20
Time (s)

b